Sound of Culture
Report metodologico sulla pipeline country-only, adjacent e Ultimate Guitar

Nota tecnica di replica cold-start

April 17, 2026

Contents

1 Obiettivo del report **3**

2 Panoramica generale **3**

3 Step 1: costruzione degli universi artista **4**
3.1 Logica sostantiva 4
3.2 Come vengono riempiti i campi anagrafici gia nel builder base 4

4 Step 2: recupero residuo dei missing in `artist_universe_country_only` **10**
4.1 Perche esiste un secondo passaggio 10
4.2 Ordine operativo del top-up 11
4.3 Logica dettagliata per variabile 17

5 Step 3: discovery e download da Ultimate Guitar **18**
5.1 Perche il download e separato in discovery, plan e download 18

6 Step 4: supplemento Billboard **22**
6.1 Perche serve un supplemento 22

7 Step 5: costruzione del dataset finale chord-level **25**
7.1 Meccanica del builder finale 25

8 Step 6: recupero dei `release_year` mancanti **28**
8.1 Gerarchia di imputazione 28

9 Replica cold-start e replica da snapshot **32**
9.1 Cold-start reale 32
9.2 Replica assistita da snapshot 33

10 Come leggere correttamente il rapporto tra country-only, adjacent e dataset finale **34**

11 Conclusione operativa **34**

A Dove trovare il codice completo **35**

1 Obiettivo del report

Questo secondo report non commenta i trend descrittivi del dataset finale, gia documentati nel report esplorativo, ma ricostruisce in modo operativo l'intera pipeline che porta dal perimetro degli artisti country alla costruzione del dataset chord-level finale. L'obiettivo e duplice: da un lato lasciare una traccia leggibile di *che cosa e stato fatto davvero*; dall'altro rendere la pipeline replicabile sia in modalita snapshot-assisted sia in modalita cold-start, cioe rifacendo gli step chiave a partire dagli script sorgente.

Il punto centrale da chiarire e che il progetto ha piu livelli logici:

1. un universo artisti, costruito prima a livello di `country-only`, poi esteso a `adjacent-only` e all'unione completa;
2. una pipeline di discovery e download su Ultimate Guitar che usa `artist_universe_country_only.csv` come input;
3. una fase di supplemento Billboard per artisti o canzoni ancora mancanti;
4. un builder finale che parte dal file `Restricted Final v6`, aggiunge nuove canzoni UG, riattacca i metadati artista, costruisce i flag Billboard e riempie i `release_year` mancanti.

Questa architettura spiega anche perche nel repository coesistono sia script di costruzione "live" sia wrapper di replica che ripristinano snapshot e cache. La replica completa, in pratica, e una combinazione tra codice, checkpoint intermedi e output consolidati.

2 Panoramica generale

Table 1: Key scripts in the country-only pipeline

Script	Role
build_country_artists_dataset.py	Builds the base country-only, adjacent-only and union ar
enrich_artist_universe_missing_metadata.py	Top-up pass for residual missing birth metadata in artis
country_only_ultimate_guitar.py	Discovery, planning and raw JSON download from Ultin
supplement_billboard_country_chords.py	Adds Billboard artists and missing chart songs not covered by b
build_country_only_chords_final.py	Merges v6 base, new UG chord rows, chart flags and releas
run_full_replication.py	Cold-start or snapshot-assisted replication wrapper for co
run_country_songs_replication.py	Restores or snapshots the upstream country-song deliverab
run_country_merge_v6_replication.py	Restores or rebuilds the restricted final v6 bridge used

Table 2: Current main artifacts produced by the pipeline

Artifact	Rows / objects	Comment
Base country-only artist universe	2,333	Country-union core before Billb
Adjacent-only artist universe	2,493	Adjacent-genre expansion kept separate fr
Full artist universe	4,824	Country core plus adjacent-
Current artist_universe_country_only.csv	2,539	Current country-only file after Billboard additions
Current new chord-song cache	36,584	Songs prepared for append after deduplicati
Current final country-only chord dataset	44,058	Final chord-level dataset used in the

La fotografia corrente degli artefatti chiave e la seguente:

- il nucleo storico `country-only` contiene 2,333 artisti;
- l'universo `adjacent-only` aggiunge 2,493 artisti non nel nucleo ma rilevanti come perimetro esteso;
- l'unione completa degli universi artista arriva a 4,824 record;
- il file `artist_universe_country_only.csv` attualmente in uso contiene 2,539 record, cioe il nucleo country piu 206 artisti aggiunti nel supplemento Billboard;
- il dataset finale `Sound_of_Culture_Country_CountryOnly_Chords_Final_2026_04_08.csv` contiene 44,058 osservazioni;
- il file intermedio `country_only_chords_new_songs_2026_04_08.csv` contiene 36,584 righe gia pronte per l'append al v6.

La relazione tra questi oggetti e importante: l'universo artisti non e solo un dizionario anagrafico, ma e la chiave che decide quali ricerche fare su UG, quali artisti hanno diritto di entrare nella fase song-level e come attaccare al file finale i metadati demografici recuperati nelle fasi precedenti.

3 Step 1: costruzione degli universi artista

3.1 Logica sostantiva

La pipeline degli artisti nasce in `build_country_artists_dataset.py`. Lo script raccoglie seed da Hall of Fame, categorie e liste Wikipedia, amplia il perimetro con query Wikidata sui generi country-correlati, poi standardizza e deduplica. La scelta metodologica qui e chiara: prima si costruisce un universo artista con un criterio relativamente conservativo ma ricco di metadati, poi si usa quell'universo come spina dorsale di tutto il resto.

Nel report QC salvato dallo script, il perimetro iniziale e:

- 2,333 artisti nel `core country-union`;
- 2,493 artisti `adjacent-only`;
- 4,824 artisti nel full artist universe.

Il fatto che il file `artist_universe_country_only.csv` corrente sia piu grande del nucleo originario non e un errore: e l'effetto della fase successiva di supplemento Billboard, che aggiunge artisti utili per coprire canzoni charted ancora assenti.

3.2 Come vengono riempiti i campi anagrafici gia nel builder base

Il builder base recupera prima metadati strutturati da Wikidata, poi li completa con Wikipedia e, solo dove serve, con MusicBrainz. La funzione piu importante e `prepare_dataset()`, che:

1. unisce i seed con i dettagli Wikidata;
2. riempie gli URL Wikipedia mancanti dalle sitelinks;
3. fa enrichment da infobox e lead Wikipedia;

4. costruisce `birth_year` da `birth_date` o da indizi secondari;
5. risolve `birth_city`, `birth_state`, `birth_country`;
6. applica un fallback da categorie Wikipedia se la geografia e ancora incompleta;
7. usa un enrichment MusicBrainz mirato sui casi ancora incompleti;
8. costruisce score di rilevanza country, flag di campione, confidence e note.

Snippet chiave: orchestrazione del builder artista.

```
def prepare_dataset(seed_df: pd.DataFrame, detail_df: pd.DataFrame, s: requests.Session) -> pd.DataFrame:
    seed_summary = (
        seed_df.groupby("wikidata_qid", dropna=False)
        .agg(
            source_seed=("source_seed", lambda items: pipe_join(items)),
            seed_title=("seed_title", "first"),
        )
        .reset_index()
    )
    merged = detail_df.merge(seed_summary, on="wikidata_qid", how="left")
    merged["source_seed"] = merged["source_seed"].fillna("Wikidata␣genre␣query")
    merged = fill_missing_wikipedia_urls_from_sitelinks(s, merged)
    merged = enrich_from_wikipedia(s, merged, INTERMEDIATE_DIR / "country_artists_wikipedia_enrichment.csv")
    merged["name_primary"] = merged.apply(choose_name_primary, axis=1)
    merged["birth_date"] = merged["birth_date"].map(parse_iso_date)
    merged["death_date"] = merged["death_date"].map(parse_iso_date)
    merged["birth_date"] = merged["birth_date"].fillna("").astype(str).replace({"NaT": "", "nan": ""})
    merged["death_date"] = merged["death_date"].fillna("").astype(str).replace({"NaT": "", "nan": ""})
    merged["birth_year"] = merged["birth_date"].str.slice(0, 4).map(maybe_int)

    geo_df = merged.apply(resolve_birth_geography, axis=1, result_type="expand")
    merged = pd.concat([merged, geo_df], axis=1)
    merged = enrich_from_wikipedia_lead(s, merged, INTERMEDIATE_DIR / "country_artists_wikipedia_lead.csv")
    merged["birth_date"] = merged["birth_date"].map(parse_iso_date)
    merged["birth_date"] = merged["birth_date"].fillna("").astype(str).replace({"NaT": "", "nan": ""})
    merged["birth_year_hint"] = merged.apply(
        lambda row: maybe_int(clean_text(row["birth_date"])[:4])
        or row.get("birth_year")
        or extract_year_hint(row.get("wp_birth_year", ""))
        or extract_birth_year_from_categories(row.get("wp_categories", "")),
        axis=1,
    )
    geo_df = merged.apply(resolve_birth_geography, axis=1, result_type="expand")
    merged = merged.drop(columns=["birth_city", "birth_county", "birth_state", "birth_state_abbr", "
     birth_country"], errors="ignore")
    merged = pd.concat([merged, geo_df], axis=1)
    geo_fallback_df = merged.apply(apply_category_location_fallback, axis=1, result_type="expand")
    for col in ["birth_city", "birth_county", "birth_state", "birth_state_abbr", "birth_country"]:
        merged[col] = geo_fallback_df[col]

    merged["birth_year"] = merged.apply(
        lambda row: maybe_int(clean_text(row["birth_date"])[:4]) if clean_text(row["birth_date"]) else row["
     birth_year_hint"],
        axis=1,
    )
    merged["death_year"] = merged["death_date"].str.slice(0, 4).map(maybe_int)
    merged["birth_decade"] = merged["birth_year"].map(lambda x: int(math.floor(x / 10) * 10) if pd.notna(x)
     else None)
    merged["us_macro_region"] = merged["birth_state"].map(lambda x: STATE_TO_REGION.get(x, "Unknown"))
    merged["is_deceased"] = merged["death_date"].map(lambda x: 1 if x else 0)

    today = date.today()

    def calc_age(row: pd.Series) -> Optional[int]:
        if not row["birth_date"]:
```

```
        return None
    try:
        born = datetime.fromisoformat(row["birth_date"]).date()
        ended = datetime.fromisoformat(row["death_date"]).date() if row["death_date"] else today
        return ended.year - born.year - ((ended.month, ended.day) < (born.month, born.day))
    except Exception:
        return None

merged["notes"] = ""
merged = targeted_musicbrainz_bio_enrichment(s, merged, INTERMEDIATE_DIR / "
 country_artists_musicbrainz_bio.csv")
merged["birth_year"] = merged.apply(
    lambda row: maybe_int(clean_text(row["birth_date"])[:4]) if clean_text(row["birth_date"]) else row["
 birth_year"],
    axis=1,
)
merged["age_or_age_at_death"] = merged.apply(calc_age, axis=1)
merged["genres_normalized"] = merged["genres_raw"].map(normalize_country_genres)
merged["instruments"] = merged["instruments"].map(lambda x: normalize_values(str(x),
 INSTRUMENT_NORMALIZATION))
merged["country_relevance_score"] = merged.apply(
    lambda row: score_country_relevance(
        row["genres_raw"],
        row["occupations"],
        row["source_seed"],
        row["awards"],
        row.get("wp_categories", ""),
    ),
    axis=1,
)
merged["birth_date_confidence"] = merged.apply(
    lambda row: "high" if row["birth_date"] else ("medium" if pd.notna(row["birth_year"]) else "low"),
    axis=1,
)
merged["birth_place_confidence"] = merged.apply(
    lambda row: "high" if row["birth_state"] and row["birth_place_raw"] else ("medium" if row["birth_state
 "] else "low"),
    axis=1,
)
merged["genre_confidence"] = merged["country_relevance_score"].map(
    lambda x: "high" if x >= 0.8 else ("medium" if x >= 0.5 else "low")
)
merged["manual_review_needed"] = merged.apply(
    lambda row: 1
    if (
        not row["wikidata_qid"]
        or pd.isna(row["birth_year"])
        or (row["birth_country"] == "United␣States" and not row["birth_state"])
        or (row["genre_confidence"] == "low" and not has_strong_country_seed_evidence(row["source_seed"],
 row.get("wp_categories", "")))
    )
    else 0,
    axis=1,
)
merged["source_primary"] = "Wikidata"
merged["source_secondary"] = ""
merged["evidence_urls"] = merged.apply(
    lambda row: pipe_join(
        [
            f"https://www.wikidata.org/wiki/{row['wikidata_qid']}" if row["wikidata_qid"] else "",
            row["wikipedia_url"],
            f"https://musicbrainz.org/artist/{row['musicbrainz_mbid']}" if clean_text(row["
 musicbrainz_mbid"]) else "",
        ]
    ),
    axis=1,
)
```

```
    merged["source_count"] = merged["evidence_urls"].map(lambda x: len([piece for piece in str(x).split("|")
     if piece]))
    merged["notes"] = merged.apply(
        lambda row: pipe_join(
            [
                clean_text(row.get("notes", "")),
                "birth_state␣unresolved␣from␣structured␣place␣hierarchy"
                if row["birth_country"] == "United␣States" and not row["birth_state"]
                else "",
                "birth␣geography␣inferred␣from␣Wikipedia␣categories"
                if row["birth_country"] == "United␣States" and row["birth_state"] and not row["birth_place_raw
     "]
                else "",
                "low␣country␣confidence" if row["genre_confidence"] == "low" else "",
            ]
        ),
        axis=1,
    )
    sample_df = merged.apply(infer_samples, axis=1, result_type="expand")
    merged = pd.concat([merged, sample_df], axis=1)
    merged["artist_id"] = [f"artist_{index:06d}" for index in range(1, len(merged) + 1)]
    merged = merged.drop(columns=["birth_year_hint"], errors="ignore")
    if "seed_title" in merged.columns:
        merged = merged.drop(columns=["seed_title"])
    return merged
```

Snippet chiave: risoluzione della geografia di nascita.

```
def resolve_birth_geography(row: pd.Series) -> dict:
    birth_admin_labels = "" if pd.isna(row.get("birth_admin_labels", "")) else str(row.get("birth_admin_labels
     ", ""))
    labels = [piece.strip() for piece in birth_admin_labels.split("|") if piece.strip()]
    raw = str(row.get("birth_place_raw", "")).strip()
    birth_state = ""
    birth_country = ""
    birth_city = ""
    birth_county = ""

    if raw:
        birth_city = raw.split(",")[0].strip()
        if COUNTY_PATTERN.search(raw):
            birth_county = raw.split(",")[0].strip()

    for label in [raw] + labels:
        for state in US_STATE_ABBR:
            if re.search(rf"\b{re.escape(state)}\b", label):
                birth_state = state
                break
        if birth_state:
            break

    raw_country = str(row.get("birth_country_raw", "")).strip()
    combined_geo = "|".join([raw, birth_admin_labels, raw_country])
    if "United␣States" in combined_geo or "USA" in combined_geo or birth_state:
        birth_country = "United␣States"
    elif raw_country:
        birth_country = raw_country

    for label in labels:
        if COUNTY_PATTERN.search(label):
            birth_county = label
            break

    return {
        "birth_city": birth_city if birth_city and birth_city != birth_county else birth_city,
        "birth_county": birth_county,
        "birth_state": birth_state,
```

```
        "birth_state_abbr": US_STATE_ABBR.get(birth_state, ""),
        "birth_country": birth_country,
    }


def apply_category_location_fallback(row: pd.Series) -> dict:
    inferred = infer_us_location_from_categories(row.get("wp_categories", ""), row.get("source_seed", ""))
    birth_city = clean_text(row.get("birth_city", "")) or clean_text(inferred["birth_city_from_categories"])
    birth_county = clean_text(row.get("birth_county", "")) or clean_text(inferred["
     birth_county_from_categories"])
    birth_state = clean_text(row.get("birth_state", "")) or clean_text(inferred["birth_state_from_categories"
     ])
    birth_country = clean_text(row.get("birth_country", "")) or clean_text(inferred["
     birth_country_from_categories"])
    if not birth_country and birth_state:
        birth_country = "United␣States"
    return {
        "birth_city": birth_city,
        "birth_county": birth_county,
        "birth_state": birth_state,
        "birth_state_abbr": US_STATE_ABBR.get(birth_state, ""),
        "birth_country": birth_country,
    }
```

Snippet chiave: enrichment MusicBrainz mirato nel builder base.

```
def targeted_musicbrainz_bio_enrichment(s: requests.Session, df: pd.DataFrame, checkpoint_path: Path) -> pd.
     DataFrame:
    needs = df[
        df["musicbrainz_mbid"].map(lambda x: not is_blank(x))
        & (
            df["birth_year"].isna()
            | df["birth_city"].map(is_blank)
            | df["birth_state"].map(is_blank)
            | df["birth_country"].map(is_blank)
        )
    ].copy()
    if needs.empty:
        return df

    if checkpoint_path.exists():
        checkpoint_df = pd.read_csv(checkpoint_path)
    else:
        checkpoint_df = pd.DataFrame(
            columns=[
                "musicbrainz_mbid",
                "mb_birth_date",
                "mb_birth_year",
                "mb_birth_city",
                "mb_birth_county",
                "mb_birth_state",
                "mb_birth_country",
            ]
        )
    for column in [
        "musicbrainz_mbid",
        "mb_birth_date",
        "mb_birth_year",
        "mb_birth_city",
        "mb_birth_county",
        "mb_birth_state",
        "mb_birth_country",
    ]:
        if column not in checkpoint_df.columns:
            checkpoint_df[column] = ""

    done = set(checkpoint_df["musicbrainz_mbid"].dropna().astype(str).tolist())
```

```
pending = needs[~needs["musicbrainz_mbid"].isin(done)].copy()
log(f"Targeted␣MusicBrainz␣bio␣enrichment␣pending␣for␣{len(pending)}␣records")

area_cache: Dict[str, dict] = {}
new_rows = []
for index, (_, row) in enumerate(pending.iterrows(), start=1):
    mbid = clean_text(row["musicbrainz_mbid"])
    try:
        payload = request_json(s, f"{MUSICBRAINZ_API}/artist/{mbid}", params={"fmt": "json"}, pause=1.05)
        begin_area = payload.get("begin-area", {}) or {}
        begin_area_name = clean_text(begin_area.get("name", ""))
        begin_area_type = clean_text(begin_area.get("type", ""))
        begin_area_id = clean_text(begin_area.get("id", ""))
        area = payload.get("area", {}) or {}
        area_name = clean_text(area.get("name", ""))
        area_state = area_name if area_name in US_STATE_ABBR else ""
        area_country = "United␣States" if clean_text(payload.get("country", "")) == "US" else ""
        area_ctx = musicbrainz_area_context(s, begin_area_id, area_cache) if begin_area_id else {
            "mb_birth_city": "",
            "mb_birth_county": "",
            "mb_birth_state": "",
            "mb_birth_country": "",
        }
        mb_birth_date = parse_iso_date(clean_text(payload.get("life-span", {}).get("begin", "")))
        mb_birth_year = extract_year_hint(clean_text(payload.get("life-span", {}).get("begin", "")) or ""
        mb_birth_city = ""
        mb_birth_county = area_ctx["mb_birth_county"]
        mb_birth_state = area_ctx["mb_birth_state"] or area_state
        mb_birth_country = area_ctx["mb_birth_country"] or area_country

        if begin_area_type == "City" and begin_area_name:
            mb_birth_city = begin_area_name
        elif begin_area_name and "County" in begin_area_name:
            mb_birth_county = mb_birth_county or begin_area_name

        new_rows.append(
            {
                "musicbrainz_mbid": mbid,
                "mb_birth_date": mb_birth_date,
                "mb_birth_year": mb_birth_year,
                "mb_birth_city": mb_birth_city,
                "mb_birth_county": mb_birth_county,
                "mb_birth_state": mb_birth_state,
                "mb_birth_country": mb_birth_country,
            }
        )
    except Exception as exc:
        new_rows.append(
            {
                "musicbrainz_mbid": mbid,
                "mb_birth_date": "",
                "mb_birth_year": "",
                "mb_birth_city": "",
                "mb_birth_county": "",
                "mb_birth_state": "",
                "mb_birth_country": "",
            }
        )
        log(f"Targeted␣MusicBrainz␣enrichment␣failed␣for␣{mbid}:␣{exc}")

    if new_rows and (index % 25 == 0 or index == len(pending)):
        checkpoint_df = pd.concat([checkpoint_df, pd.DataFrame(new_rows)], ignore_index=True)
        checkpoint_df = checkpoint_df.drop_duplicates(subset=["musicbrainz_mbid"], keep="last")
        checkpoint_df.to_csv(checkpoint_path, index=False)
        log(f"Saved␣MusicBrainz␣bio␣checkpoint␣at␣{len(checkpoint_df)}␣rows")
        new_rows = []

enriched = df.merge(checkpoint_df, on="musicbrainz_mbid", how="left")
```

```
    enriched["birth_date"] = enriched.apply(
        lambda row: clean_text(row["birth_date"]) or clean_text(row["mb_birth_date"]),
        axis=1,
    )
    enriched["birth_year"] = enriched.apply(
        lambda row: row["birth_year"] if pd.notna(row["birth_year"]) else extract_year_hint(row.get("
     mb_birth_year", "")),
        axis=1,
    )
    enriched["birth_city"] = enriched.apply(
        lambda row: clean_text(row["birth_city"]) or clean_text(row["mb_birth_city"]),
        axis=1,
    )
    enriched["birth_county"] = enriched.apply(
        lambda row: clean_text(row["birth_county"]) or clean_text(row["mb_birth_county"]),
        axis=1,
    )
    enriched["birth_state"] = enriched.apply(
        lambda row: clean_text(row["birth_state"]) or clean_text(row["mb_birth_state"]),
        axis=1,
    )
    enriched["birth_country"] = enriched.apply(
        lambda row: clean_text(row["birth_country"]) or clean_text(row["mb_birth_country"]),
        axis=1,
    )
    enriched["birth_state_abbr"] = enriched["birth_state"].map(lambda x: US_STATE_ABBR.get(clean_text(x), ""))
    enriched["notes"] = enriched.apply(
        lambda row: pipe_join(
            [
                clean_text(row.get("notes", "")),
                "birth␣data␣enriched␣from␣MusicBrainz"
                if any(
                    clean_text(row.get(col, ""))
                    for col in ["mb_birth_date", "mb_birth_city", "mb_birth_county", "mb_birth_state", "
     mb_birth_country"]
                )
                else "",
            ]
        ),
        axis=1,
    )
    return enriched.drop(
        columns=[
            "mb_birth_date",
            "mb_birth_year",
            "mb_birth_city",
            "mb_birth_county",
            "mb_birth_state",
            "mb_birth_country",
        ]
    )
```

La cosa importante da leggere in questi snippet e l'ordine. Il progetto non fa un'imputazione cieca: parte da campi strutturati forti, usa il lead Wikipedia per colmare lacune narrative, poi applica fallback espliciti e annotabili. Questa gerarchia e una scelta di qualita del dato, non solo di convenienza tecnica.

4 Step 2: recupero residuo dei missing in `artist_universe_country_only`

4.1 Perche esiste un secondo passaggio

Il builder base dell'universo artista e pensato per la costruzione generale degli output `country_artists_*`. Ma il file operativo usato a valle dalla pipeline UG e `artist_universe_country_only.csv`. Per

questo esiste uno script separato, `enrich_artist_universe_missing_metadata.py`, che fa un *top-up* diretto dei missing residui sul file country-only.

Questa distinzione e cruciale per la replica:

- `build_country_artists_dataset.py` costruisce l'universo artisti generale e i campioni principali;
- `enrich_artist_universe_missing_metadata.py -country-only-direct` rifinisce il file country-only effettivamente usato dagli step successivi.

4.2 Ordine operativo del top-up

La funzione `enrich_country_only_direct()` documenta esplicitamente la sequenza del recupero residuo:

1. parsing e pulizia della geografia gia presente;
2. risoluzione dei QID Wikidata mancanti dai nomi;
3. enrichment mirato da QID Wikidata;
4. enrichment da Wikipedia;
5. risoluzione dei MBID MusicBrainz mancanti;
6. enrichment MusicBrainz mirato;
7. enrichment MusicBrainz generico;
8. recupero di anno/origine da Wikipedia per i gruppi o i casi residui;
9. enrichment Discogs e iTunes/Apple Music;
10. enrichment dagli official websites via schema JSON-LD;
11. applicazione di override manuali web-confirmed;
12. nuova pulizia geografica e inferenza dello stato da coppie citta-paese gia osservate;
13. refresh delle variabili derivate e salvataggio finale del country-only.

Snippet chiave: pipeline completa di top-up sul country-only.

```
def enrich_country_only_direct() -> None:
    path = OUTPUT_DIR / "artist_universe_country_only.csv"
    df = collapse_merge_columns(pd.read_csv(path, low_memory=False))
    report_missingness(df, "country_only␣before␣enrichment")

    df = apply_place_parsing(df)
    df = clean_birth_geography(df)
    df = resolve_missing_qids_from_names(df, "artist_universe_country_only_name_to_qid.csv")
    df = enrich_from_wikidata_qids(df, "artist_universe_country_only_targeted_wikidata.csv")
    df = enrich_from_wikipedia_sources(df, "country_artists")
    df = resolve_missing_musicbrainz_mbids(df, "artist_universe_country_only_name_to_musicbrainz.csv")
    df = builder.targeted_musicbrainz_bio_enrichment(builder.session(), df, INTERMEDIATE_DIR / "
     artist_universe_country_only_targeted_musicbrainz_bio.csv")
    df = enrich_from_musicbrainz_generic(df, "artist_universe_country_only_generic_musicbrainz.csv")
    df = enrich_from_wikipedia_origin_years(df, "artist_universe_country_only_wikipedia_origin_years.csv")
    df = enrich_with_discogs_and_itunes(df, "artist_universe_country_only_discogs_itunes.csv")
    df = enrich_from_official_websites(df, "artist_universe_country_only_official_websites.csv")
```

```
    df = apply_web_confirmed_overrides(df)
    df = apply_group_and_occupation_annotations(df)
    df = apply_place_parsing(df)
    df = clean_birth_geography(df)
    df = infer_state_from_existing_city_country(df)
    df = refresh_country_only_derived_columns(df)
    report_missingness(df, "country_only␣after␣enrichment")
    save_country_only_dataset(df)
```

Snippet chiave: risoluzione dei QID mancanti dai nomi.

```
def resolve_missing_qids_from_names(df: pd.DataFrame, checkpoint_name: str) -> pd.DataFrame:
    enriched = df.copy()
    needs = enriched[
        enriched["wikidata_qid"].fillna("").astype(str).str.strip().eq("")
        & (
            enriched["birth_year"].isna()
            | enriched["birth_state"].fillna("").astype(str).str.strip().eq("")
            | enriched["birth_country"].fillna("").astype(str).str.strip().eq("")
            | enriched["birth_city"].fillna("").astype(str).str.strip().eq("")
        )
    ].copy()
    if needs.empty:
        return enriched

    checkpoint_path = INTERMEDIATE_DIR / checkpoint_name
    if checkpoint_path.exists():
        checkpoint_df = pd.read_csv(checkpoint_path)
    else:
        checkpoint_df = pd.DataFrame(columns=["name_primary", "resolved_wikidata_qid"])

    done = set(checkpoint_df["name_primary"].fillna("").astype(str))
    pending_names = [name for name in needs["name_primary"].fillna("").astype(str).tolist() if name and name
     not in done]
    if pending_names:
        s = builder.session()
        new_rows: List[dict] = []
        for batch_start in range(0, len(pending_names), 20):
            batch = pending_names[batch_start : batch_start + 20]
            try:
                batch_map = builder.wikipedia_titles_to_qids(s, batch)
            except Exception as exc:
                builder.log(f"Batch␣Wikipedia␣title-to-QID␣lookup␣failed␣for␣{len(batch)}␣names:␣{exc}")
                batch_map = {name: "" for name in batch}
            for name in batch:
                new_rows.append({"name_primary": name, "resolved_wikidata_qid": batch_map.get(name, "") or ""
     })
            checkpoint_df = pd.concat([checkpoint_df, pd.DataFrame(new_rows)], ignore_index=True)
            checkpoint_df = checkpoint_df.drop_duplicates(subset=["name_primary"], keep="last")
            checkpoint_df.to_csv(checkpoint_path, index=False)
            builder.log(f"Saved␣name-to-QID␣checkpoint␣at␣{len(checkpoint_df)}␣rows")
            new_rows = []

        unresolved_mask = checkpoint_df["resolved_wikidata_qid"].fillna("").astype(str).str.strip().eq("")
        unresolved_names = checkpoint_df.loc[unresolved_mask, "name_primary"].dropna().astype(str).tolist()
        consecutive_search_failures = 0
        for index, name in enumerate(unresolved_names, start=1):
            try:
                qid = builder.search_wikidata_qid(s, name) or ""
                consecutive_search_failures = 0
            except Exception as exc:
                builder.log(f"Wikidata␣title␣search␣failed␣for␣{name}:␣{exc}")
                qid = ""
                consecutive_search_failures += 1
            checkpoint_df.loc[checkpoint_df["name_primary"].eq(name), "resolved_wikidata_qid"] = qid
            if index % 25 == 0 or index == len(unresolved_names):
                checkpoint_df.to_csv(checkpoint_path, index=False)
```

```
                builder.log(f"Updated␣name-to-QID␣checkpoint␣at␣{len(checkpoint_df)}␣rows")
            if consecutive_search_failures >= 10:
                builder.log("Stopping␣single-title␣QID␣search␣early␣after␣repeated␣failures/rate␣limits")
                break

    merged = enriched.merge(checkpoint_df, on="name_primary", how="left")
    merged["wikidata_qid"] = merged.apply(
        lambda row: nonempty(row.get("wikidata_qid", "")) or nonempty(row.get("resolved_wikidata_qid", "")),
        axis=1,
    )
    return merged.drop(columns=["resolved_wikidata_qid"])
```

Snippet chiave: fallback Discogs/iTunes e official websites.

```
def enrich_with_discogs_and_itunes(df: pd.DataFrame, checkpoint_name: str) -> pd.DataFrame:
    enriched = df.copy()
    needs = enriched[
        enriched["birth_year"].isna()
        | enriched["birth_state"].fillna("").astype(str).str.strip().eq("")
        | enriched["birth_country"].fillna("").astype(str).str.strip().eq("")
    ].copy()
    if needs.empty:
        return enriched

    checkpoint_path = INTERMEDIATE_DIR / checkpoint_name
    if checkpoint_path.exists():
        checkpoint_df = pd.read_csv(checkpoint_path)
    else:
        checkpoint_df = pd.DataFrame(
            columns=[
                "name_primary",
                "discogs_birth_year",
                "discogs_birth_city",
                "discogs_birth_state",
                "discogs_birth_country",
                "discogs_url",
                "itunes_artist_url",
            ]
        )

    done = set(checkpoint_df["name_primary"].fillna("").astype(str))
    pending = needs[~needs["name_primary"].isin(done)].copy()
    if pending.empty:
        merged = enriched.merge(checkpoint_df, on="name_primary", how="left")
        return apply_discogs_itunes_merge(merged)

    s = requests.Session()
    new_rows: List[dict] = []
    consecutive_discogs_failures = 0
    consecutive_itunes_failures = 0
    discogs_enabled = True
    itunes_enabled = True
    for index, (_, row) in enumerate(pending.iterrows(), start=1):
        name = nonempty(row["name_primary"])
        discogs_birth_year = None
        discogs_birth_city = ""
        discogs_birth_state = ""
        discogs_birth_country = ""
        discogs_url = ""
        itunes_artist_url = ""

        if discogs_enabled:
            try:
                search = s.get(
                    f"{DISCOGS_API}/database/search",
                    params={"q": name, "type": "artist", "per_page": 5, "page": 1},
                    headers=discogs_headers(),
```

```
                timeout=20,
            )
            search.raise_for_status()
            results = search.json().get("results", [])
            best = None
            target_key = normalize_name(name)
            for item in results:
                title_key = normalize_name(item.get("title", ""))
                if title_key == target_key:
                    best = item
                    break
            if not best and results:
                best = results[0]
            if best and best.get("resource_url"):
                artist_resp = s.get(best["resource_url"], headers=discogs_headers(), timeout=20)
                artist_resp.raise_for_status()
                artist_payload = artist_resp.json()
                parsed = parse_discogs_profile(artist_payload.get("profile", ""))
                discogs_birth_year = parsed["birth_year"]
                discogs_birth_city = parsed["birth_city"]
                discogs_birth_state = parsed["birth_state"]
                discogs_birth_country = parsed["birth_country"]
                discogs_url = nonempty(best.get("uri", ""))
            consecutive_discogs_failures = 0
        except Exception as exc:
            builder.log(f"Discogs␣enrichment␣failed␣for␣{name}:␣{exc}")
            consecutive_discogs_failures += 1
            if consecutive_discogs_failures >= 10:
                discogs_enabled = False
                builder.log("Discogs␣enrichment␣disabled␣for␣the␣rest␣of␣this␣run␣after␣repeated␣failures/
rate␣limits")

    if itunes_enabled:
        try:
            itunes = s.get(
                ITUNES_SEARCH_API,
                params={"term": name, "entity": "musicArtist", "attribute": "artistTerm", "limit": 5},
                timeout=20,
            )
            itunes.raise_for_status()
            results = itunes.json().get("results", [])
            best = None
            target_key = normalize_name(name)
            for item in results:
                title_key = normalize_name(item.get("artistName", ""))
                if title_key == target_key:
                    best = item
                    break
            if not best and results:
                best = results[0]
            if best:
                itunes_artist_url = nonempty(best.get("artistLinkUrl", ""))
            consecutive_itunes_failures = 0
        except Exception as exc:
            builder.log(f"iTunes/Apple␣Music␣lookup␣failed␣for␣{name}:␣{exc}")
            consecutive_itunes_failures += 1
            if consecutive_itunes_failures >= 10:
                itunes_enabled = False
                builder.log("iTunes/Apple␣Music␣enrichment␣disabled␣for␣the␣rest␣of␣this␣run␣after␣
repeated␣failures/rate␣limits")

    new_rows.append(
        {
            "name_primary": name,
            "discogs_birth_year": discogs_birth_year,
            "discogs_birth_city": discogs_birth_city,
            "discogs_birth_state": discogs_birth_state,
            "discogs_birth_country": discogs_birth_country,
```

```
                "discogs_url": discogs_url,
                "itunes_artist_url": itunes_artist_url,
            }
        )

        if new_rows and (index % 25 == 0 or index == len(pending)):
            checkpoint_df = pd.concat([checkpoint_df, pd.DataFrame(new_rows)], ignore_index=True)
            checkpoint_df = checkpoint_df.drop_duplicates(subset=["name_primary"], keep="last")
            checkpoint_df.to_csv(checkpoint_path, index=False)
            builder.log(f"Saved␣Discogs/iTunes␣checkpoint␣at␣{len(checkpoint_df)}␣rows")
            new_rows = []

    merged = enriched.merge(checkpoint_df, on="name_primary", how="left")
    return apply_discogs_itunes_merge(merged)

def enrich_from_official_websites(df: pd.DataFrame, checkpoint_name: str) -> pd.DataFrame:
    enriched = df.copy()
    needs = enriched[
        enriched["official_website"].fillna("").astype(str).str.strip().ne("")
        & (
            enriched["birth_year"].isna()
            | enriched["birth_state"].fillna("").astype(str).str.strip().eq("")
            | enriched["birth_city"].fillna("").astype(str).str.strip().eq("")
            | enriched["birth_country"].fillna("").astype(str).str.strip().eq("")
        )
    ].copy()
    if needs.empty:
        return enriched

    checkpoint_path = INTERMEDIATE_DIR / checkpoint_name
    if checkpoint_path.exists():
        checkpoint_df = pd.read_csv(checkpoint_path)
    else:
        checkpoint_df = pd.DataFrame(
            columns=[
                "official_website",
                "official_birth_year",
                "official_birth_city",
                "official_birth_county",
                "official_birth_state",
                "official_birth_country",
                "official_schema_url",
            ]
        )

    done = set(checkpoint_df["official_website"].fillna("").astype(str))
    pending = needs[~needs["official_website"].isin(done)].copy()
    new_rows: List[dict] = []
    for index, (_, row) in enumerate(pending.iterrows(), start=1):
        url = nonempty(row["official_website"])
        parsed = {
            "official_birth_year": None,
            "official_birth_city": "",
            "official_birth_county": "",
            "official_birth_state": "",
            "official_birth_country": "",
            "official_schema_url": "",
        }
        try:
            parsed = parse_official_website_schema(url, is_group_entity(row))
        except Exception as exc:
            builder.log(f"Official␣website␣schema␣enrichment␣failed␣for␣{url}:␣{exc}")
        new_rows.append({"official_website": url, **parsed})
        if new_rows and (index % 25 == 0 or index == len(pending)):
            checkpoint_df = pd.concat([checkpoint_df, pd.DataFrame(new_rows)], ignore_index=True)
            checkpoint_df = checkpoint_df.drop_duplicates(subset=["official_website"], keep="last")
            checkpoint_df.to_csv(checkpoint_path, index=False)
            builder.log(f"Saved␣official-website␣checkpoint␣at␣{len(checkpoint_df)}␣rows")
```

```
        new_rows = []

    merged = enriched.merge(checkpoint_df, on="official_website", how="left")
    merged["birth_year"] = merged.apply(
        lambda row: row["birth_year"] if pd.notna(row["birth_year"]) else builder.extract_year_hint(row.get("
     official_birth_year", "")),
        axis=1,
    )
    for source_col, target_col in [
        ("official_birth_city", "birth_city"),
        ("official_birth_county", "birth_county"),
        ("official_birth_state", "birth_state"),
        ("official_birth_country", "birth_country"),
    ]:
        merged[target_col] = merged.apply(
            lambda row: nonempty(row.get(target_col, "")) or nonempty(row.get(source_col, "")),
            axis=1,
        )
    merged["evidence_urls"] = merged.apply(
        lambda row: "|".join(piece for piece in [nonempty(row.get("evidence_urls", "")), nonempty(row.get("
     official_schema_url", ""))] if piece),
        axis=1,
    )
    merged["notes"] = merged.apply(
        lambda row: "|".join(
            piece
            for piece in [nonempty(row.get("notes", "")), "birth/origin␣data␣enriched␣from␣official␣website␣
     schema" if nonempty(row.get("official_schema_url", "")) else ""]
            if piece
        ),
        axis=1,
    )
    merged["birth_state_abbr"] = merged["birth_state"].map(lambda value: builder.US_STATE_ABBR.get(nonempty(
     value), ""))
    drop_cols = [
        "official_birth_year",
        "official_birth_city",
        "official_birth_county",
        "official_birth_state",
        "official_birth_country",
        "official_schema_url",
    ]
    return merged.drop(columns=[col for col in drop_cols if col in merged.columns], errors="ignore")
```

Table 3: Current residual missingness in artist_universe_country_only.csv

Variable	Remaining missing in current country-only universe
birth_year	118
birth_city	178
birth_state	122
birth_country	96
wikidata_qid	89
musicbrainz_mbid	531

Questa tabella va letta con attenzione. I missing residui nel file country-only non significano necessariamente fallimento della pipeline: una parte dei record aggiunti dal supplemento Billboard riguarda gruppi, entita non statunitensi o casi con evidenza documentale piu debole. La regola pratica e che la pipeline non forza valori quando la fonte non supera una soglia minima di credibilita.

4.3 Logica dettagliata per variabile

`birth_year.` L'anno di nascita viene recuperato, in ordine, da:

1. `birth_date` strutturato;
2. campi Wikidata mirati;
3. infobox e lead Wikipedia;
4. origine/anni attivi nei gruppi;
5. Discogs;
6. official websites;
7. override manuali web-confirmed.

`birth_city` e `birth_state.` La citta e lo stato seguono una logica di parsing gerarchico:

1. parsing di `birth_place_raw`;
2. gerarchie amministrative Wikidata e MusicBrainz;
3. categorie Wikipedia e seed location fallback;
4. parsing di campi `Origin` o schema degli official websites;
5. inferenza dello stato da combinazioni citta-paese gia osservate in record completi;
6. correzioni manuali su casi noti o sospetti.

`birth_country.` Il paese viene riempito da:

1. place hierarchy strutturata;
2. country raw fields;
3. MusicBrainz area/context;
4. Discogs e official website schema;
5. override manuali.

In sintesi, il top-up non e un semplice merge di fonti, ma una pipeline gerarchica che prova prima fonti strutturate e riutilizzabili, poi fallback piu rumorosi, e lascia traccia nei `notes`.

5 Step 3: discovery e download da Ultimate Guitar

5.1 Perche il download e separato in discovery, plan e download

Lo script `country_only_ultimate_guitar.py` e costruito in tre sottocomandi distinti:

1. `discover`: cerca su UG gli artisti del country-only e salva fino a tre versioni migliori per canzone;
2. `plan`: misura quante discovery rows esistono e quante raw JSON mancano ancora sul disco;
3. `download`: scarica i payload JSON grezzi, con resume, shard opzionali e gestione di rate limits e casi 451.

Questa separazione e importante per la replica cold-start, perche consente di fermarsi e riprendere senza perdere lo stato gia acquisito.

Snippet chiave: discovery su UG con checkpoint e resume.

```
def run_discovery(max_pages, checkpoint_every, workers):
    artists = load_artists(ARTIST_CSV)
    progress = load_progress()
    processed_artist_ids = set(progress["processed_artist_ids"])
    chords_map, tabs_map = rebuild_maps(progress)

    total = len(artists)
    log(f"[discover]␣Loaded␣{total}␣artists␣from␣{ARTIST_CSV}")
    log(f"[discover]␣Resume␣state:␣{len(processed_artist_ids)}␣artists␣already␣processed")

    pending_artists = [artist for artist in artists if artist["artist_id"] not in processed_artist_ids]
    artist_positions = {artist["artist_id"]: idx for idx, artist in enumerate(artists, start=1)}

    with ThreadPoolExecutor(max_workers=workers) as executor:
        future_to_artist = {
            executor.submit(discover_artist_payload, artist_row, max_pages): artist_row
            for artist_row in pending_artists
        }
        for future in as_completed(future_to_artist):
            artist_row = future_to_artist[future]
            artist_id = artist_row["artist_id"]
            position = artist_positions[artist_id]
            log(f"[discover]␣[{position}/{total}]␣completed␣{artist_row['name_primary']}")

            try:
                _, chords_result, tabs_result = future.result()
            except Exception as exc:
                log(f"[discover]␣artist␣failed␣and␣will␣be␣retried␣on␣the␣next␣run:␣{artist_row['name_primary
    ']}␣({exc})")
                continue

            for key, versions in chords_result.items():
                chords_map[key] = versions
            for key, versions in tabs_result.items():
                tabs_map[key] = versions

            processed_artist_ids.add(artist_id)

            processed_count = len(processed_artist_ids)
            if processed_count % checkpoint_every == 0 or processed_count == total:
                save_progress(processed_artist_ids, chords_map, tabs_map, total)
                log(
                    f"[discover]␣checkpoint:␣artists={processed_count}/{total},␣"
                    f"chords_json={len(flatten_top_versions(chords_map))},␣"
                    f"tabs_json={len(flatten_top_versions(tabs_map))}"
                )
```

```
    save_progress(processed_artist_ids, chords_map, tabs_map, total)
    log(
        f"[discover]␣complete:␣artists={len(processed_artist_ids)}/{total},␣"
        f"chords_json={len(flatten_top_versions(chords_map))},␣"
        f"tabs_json={len(flatten_top_versions(tabs_map))}"
    )
```

Snippet chiave: plan, download e CLI.

```
def build_plan():
    chords = load_discovery(CHORDS_DISCOVERY_PATH)
    tabs = load_discovery(TABS_DISCOVERY_PATH)

    RAW_CHORDS_DIR.mkdir(parents=True, exist_ok=True)
    RAW_TABS_DIR.mkdir(parents=True, exist_ok=True)

    existing_chords_ids = extract_existing_ids(RAW_CHORDS_DIR)
    existing_tabs_ids = extract_existing_ids(RAW_TABS_DIR)

    chords_ids = {int(row["id"]) for row in chords}
    tabs_ids = {int(row["id"]) for row in tabs}

    plan = {
        "generated_at_utc": datetime.now(timezone.utc).isoformat(),
        "artists_targeted": len(load_artists(ARTIST_CSV)),
        "chords": {
            "discovery_file": str(CHORDS_DISCOVERY_PATH),
            "output_dir": str(RAW_CHORDS_DIR),
            "unique_songs": count_unique_songs(chords),
            "json_total": len(chords_ids),
            "json_existing": len(chords_ids & existing_chords_ids),
            "json_missing": len(chords_ids - existing_chords_ids),
        },
        "tabs": {
            "discovery_file": str(TABS_DISCOVERY_PATH),
            "output_dir": str(RAW_TABS_DIR),
            "unique_songs": count_unique_songs(tabs),
            "json_total": len(tabs_ids),
            "json_existing": len(tabs_ids & existing_tabs_ids),
            "json_missing": len(tabs_ids - existing_tabs_ids),
        },
    }
    plan["combined"] = {
        "json_total": plan["chords"]["json_total"] + plan["tabs"]["json_total"],
        "json_existing": plan["chords"]["json_existing"] + plan["tabs"]["json_existing"],
        "json_missing": plan["chords"]["json_missing"] + plan["tabs"]["json_missing"],
    }

    with open(PLAN_PATH, "w", encoding="utf-8") as handle:
        json.dump(plan, handle, indent=2)

    print(json.dumps(plan, indent=2), flush=True)


def download_rows(rows, output_dir, workers, min_delay, max_delay, shard_mod, shard_rem):
    output_dir.mkdir(parents=True, exist_ok=True)
    existing_ids = extract_existing_ids(output_dir)
    sharded_rows = rows
    if shard_mod is not None and shard_rem is not None:
        sharded_rows = [row for row in rows if int(row["id"]) % shard_mod == shard_rem]
    pending = [row for row in sharded_rows if int(row["id"]) not in existing_ids]
    total = len(rows)
    shard_total = len(sharded_rows)
    pending_total = len(pending)

    log(f"[download]␣output_dir={output_dir}")
```

```
    log(
        f"[download] discovered={total}, shard_scope={shard_total}, "
        f"existing_in_scope={shard_total - pending_total}, pending={pending_total}"
    )

    if not pending:
        return

    client = UltimateGuitarClient()
    shared_existing_ids = set(existing_ids)
    completed = 0

    def worker(row):
        nonlocal completed
        tab_id = int(row["id"])
        with counter_lock:
            if tab_id in shared_existing_ids:
                completed += 1
                return "skip"

        max_retries = 3
        for attempt in range(max_retries):
            try:
                payload = client.get_tab_info(tab_id)
                filename = (
                    f"{safe_filename_component(row.get('artist_name') or row.get('target_artist_name'))}_"
                    f"{safe_filename_component(row.get('song_name'))}_{tab_id}.json"
                )
                filepath = output_dir / filename
                with open(filepath, "w", encoding="utf-8") as handle:
                    json.dump(payload, handle, indent=2)

                with counter_lock:
                    shared_existing_ids.add(tab_id)
                    completed += 1
                    done = completed
                if done % 25 == 0 or done == pending_total:
                    log(f"[download] progress={done}/{pending_total}")
                time.sleep(random.uniform(min_delay, max_delay))
                return "success"
            except Exception as exc:
                message = str(exc)
                if "429" in message:
                    backoff = min((attempt + 1) * 30, 90)
                    log(f"[download] rate limit on {tab_id}, waiting {backoff}s")
                    time.sleep(backoff)
                    continue
                if "451" in message:
                    with counter_lock:
                        completed += 1
                    log(f"[download] legal skip 451 for {tab_id}")
                    return "legal_unavailable"
                if attempt == max_retries - 1:
                    with counter_lock:
                        completed += 1
                    log(f"[download] failed {tab_id}: {exc}")
                    return "failed"
                time.sleep(5)
        return "failed"

    with ThreadPoolExecutor(max_workers=workers) as executor:
        futures = [executor.submit(worker, row) for row in pending]
        for future in as_completed(futures):
            future.result()


def run_download(kind, workers, min_delay, max_delay, shard_mod, shard_rem):
    targets = []
```

```
    if kind in ("chords", "both"):
        targets.append(("Chords", CHORDS_DISCOVERY_PATH, RAW_CHORDS_DIR))
    if kind in ("tabs", "both"):
        targets.append(("Tabs", TABS_DISCOVERY_PATH, RAW_TABS_DIR))

    for label, discovery_path, output_dir in targets:
        rows = load_discovery(discovery_path)
        log(f"[download:{label}]␣starting")
        download_rows(
            rows,
            output_dir,
            workers,
            min_delay=min_delay,
            max_delay=max_delay,
            shard_mod=shard_mod,
            shard_rem=shard_rem,
        )
        log(f"[download:{label}]␣complete")


def parse_args():
    parser = argparse.ArgumentParser(description="Ultimate␣Guitar␣pipeline␣for␣2333␣country-only␣artists.")
    subparsers = parser.add_subparsers(dest="command", required=True)

    discover = subparsers.add_parser("discover", help="Discover␣top-3␣chords␣and␣tabs␣per␣song␣for␣all␣country
     -only␣artists.")
    discover.add_argument("--max-pages", type=int, default=20)
    discover.add_argument("--checkpoint-every", type=int, default=10)
    discover.add_argument("--workers", type=int, default=3)

    subparsers.add_parser("plan", help="Summarize␣how␣many␣JSON␣files␣exist␣and␣are␣still␣missing.")

    download = subparsers.add_parser("download", help="Download␣discovered␣JSON␣payloads.")
    download.add_argument("--kind", choices=["chords", "tabs", "both"], default="both")
    download.add_argument("--workers", type=int, default=3)
    download.add_argument("--min-delay", type=float, default=0.6)
    download.add_argument("--max-delay", type=float, default=1.6)
    download.add_argument("--shard-mod", type=int, default=None)
    download.add_argument("--shard-rem", type=int, default=None)

    return parser.parse_args()

def main():
    args = parse_args()
    if args.command == "discover":
        run_discovery(max_pages=args.max_pages, checkpoint_every=args.checkpoint_every, workers=args.workers)
    elif args.command == "plan":
        build_plan()
    elif args.command == "download":
        run_download(
            kind=args.kind,
            workers=args.workers,
            min_delay=args.min_delay,
            max_delay=args.max_delay,
            shard_mod=args.shard_mod,
            shard_rem=args.shard_rem,
        )
```

Table 4: Saved Ultimate Guitar discovery/download plan snapshot

Layer	Artists / songs / JSON	Coverage snapshot
UG chords	2,333 artists	55,335 songs; 64,829 JSONs total; 11 missing
UG tabs	2,333 artists	5,243 songs; 7,299 JSONs total; 3 missing

La lettura corretta del plan snapshot e questa:

- il progetto ha mirato a 2,333 artisti nel paese country-only base;
- per le *chords* risultano 55,335 canzoni uniche e 64,829 JSON discovery, di cui solo 11 mancanti nel plan salvato;
- per i *tabs* risultano 5,243 canzoni uniche e 7,299 JSON discovery, con 3 mancanti.

Questo spiega due fatti sostantivi. Primo, la pipeline salva piu JSON per canzone perche tiene fino a tre versioni top-rated. Secondo, il file finale chord-level non usa ciecamente un solo payload, ma puo fondere i primi tre quando servono informazioni complementari.

6 Step 4: supplemento Billboard

6.1 Perche serve un supplemento

Anche una discovery larga su UG puo lasciare fuori:

- artisti che non erano nel country-only base ma compaiono nelle liste Billboard;
- canzoni charted di artisti gia presenti ma non ancora intercettate dalla discovery larga.

Per questo esiste `supplement_billboard_country_chords.py`, che lavora su due fronti:

1. full discovery per gli artisti aggiunti dal supplemento;
2. targeted search song-by-song per canzoni Billboard ancora mancanti.

Snippet chiave: supplemento Billboard.

```
def run(
    max_pages_artist: int,
    max_pages_song: int,
    workers: int,
    min_delay: float,
    max_delay: float,
    skip_broad: bool,
    download_added_only: bool,
    target_csv: Path,
) -> None:
    universe_df = load_universe()
    artist_rows = build_artist_rows(universe_df)
    existing_rows = load_json_rows(DISCOVERY_JSON)
    existing_ids = {int(row["id"]) for row in existing_rows if row.get("id")}
    attempted_target_keys = load_progress_keys()

    added_artists_df = pd.read_csv(ADDED_ARTISTS_CSV, low_memory=False) if ADDED_ARTISTS_CSV.exists() else pd.
     DataFrame()
    targets_df = pd.read_csv(target_csv, low_memory=False)
    added_artist_ids = set(added_artists_df["artist_id"].fillna("").astype(str)) if not added_artists_df.empty
      else set()

    if download_added_only:
        rows = [row for row in existing_rows if str(row.get("artist_id", "")).strip() in added_artist_ids]
        log(f"Downloading␣raw␣JSONs␣for␣{len(rows)}␣discovery␣rows␣tied␣to␣added␣Billboard␣artists")
        base.download_rows(
            rows,
            RAW_CHORDS_DIR,
            workers=workers,
            min_delay=min_delay,
            max_delay=max_delay,
```

```
        shard_mod=None,
        shard_rem=None,
    )
    return

supplement_rows: List[dict] = []
existing_artist_ids = {str(row.get("artist_id", "")).strip() for row in existing_rows if str(row.get("
 artist_id", "")).strip()}

if not added_artists_df.empty and not skip_broad:
    pending_added_artists_df = added_artists_df[
        ~added_artists_df["artist_id"].fillna("").astype(str).isin(existing_artist_ids)
    ].copy()
    log(f"Broad␣discovery␣for␣{len(pending_added_artists_df)}␣newly␣added␣artists␣still␣missing␣from␣
 discovery")
    client = UltimateGuitarClient()
    for idx, row in enumerate(pending_added_artists_df.to_dict(orient="records"), start=1):
        artist_id = str(row.get("artist_id", "")).strip()
        artist_row = artist_rows.get(artist_id)
        if not artist_row:
            continue
        discovered = base.discover_kind_for_artist(client, artist_row, "Chords", max_pages=
 max_pages_artist)
        artist_rows_flat = []
        for versions in discovered.values():
            for version in versions:
                version["discovery_source"] = "new_artist_full"
                artist_rows_flat.append(version)
        supplement_rows.extend(artist_rows_flat)
        log(
            f"[artists]␣{idx}/{len(pending_added_artists_df)}␣{artist_row['name_primary']}␣->␣"
            f"{len(discovered)}␣songs␣/␣{len(artist_rows_flat)}␣json␣targets"
        )
        if idx % 20 == 0 or idx == len(pending_added_artists_df):
            checkpoint(existing_rows, supplement_rows)
        time.sleep(0.5)

merged_rows = merge_discovery_rows(existing_rows, supplement_rows)
known_song_keys = current_song_keys(merged_rows)

client = UltimateGuitarClient()
target_new_rows: List[dict] = []
log(f"Targeted␣discovery␣for␣{len(targets_df)}␣Billboard␣songs␣still␣missing")
for idx, row in enumerate(targets_df.to_dict(orient="records"), start=1):
    artist_id = str(row.get("artist_id", "")).strip()
    song_name = str(row.get("song_name", "")).strip()
    if not artist_id or not song_name or artist_id not in artist_rows:
        continue
    key = (artist_id, base.normalize_text(song_name))
    if key in known_song_keys:
        continue
    progress_key = f"{artist_id}::{key[1]}"
    if progress_key in attempted_target_keys:
        continue
    artist_row = {**artist_rows[artist_id], "chart_artist_name": str(row.get("chart_artist_name", "")).
 strip()}
    versions = discover_target_song(client, artist_row, song_name, max_pages=max_pages_song)
    attempted_target_keys.add(progress_key)
    if versions:
        target_new_rows.extend(versions)
        known_song_keys.add(key)
    if idx % 25 == 0 or idx == len(targets_df):
        log(f"[targets]␣progress␣{idx}/{len(targets_df)}␣|␣new␣song␣matches␣{len(target_new_rows)}")
    if idx % 50 == 0 or idx == len(targets_df):
        checkpoint(existing_rows, supplement_rows + target_new_rows)
        save_progress_keys(attempted_target_keys)
    time.sleep(0.25)
```

```
    supplement_rows.extend(target_new_rows)
    save_progress_keys(attempted_target_keys)

    final_rows = merge_discovery_rows(existing_rows, supplement_rows)
    save_json_rows(DISCOVERY_JSON, final_rows)
    save_json_rows(SUPPLEMENT_DISCOVERY_JSON, supplement_rows)
    log(
        f"Merged␣discovery␣saved:␣existing={len(existing_rows)},␣supplement={len(supplement_rows)},␣"
        f"final={len(final_rows)}"
    )

    final_ids = {int(row["id"]) for row in final_rows if row.get("id")}
    new_ids = sorted(final_ids - existing_ids)
    download_rows = [row for row in final_rows if int(row.get("id") or 0) in set(new_ids)]
    log(f"Downloading␣{len(download_rows)}␣newly␣added␣chord␣JSON␣payloads")
    base.download_rows(
        download_rows,
        RAW_CHORDS_DIR,
        workers=workers,
        min_delay=min_delay,
        max_delay=max_delay,
        shard_mod=None,
        shard_rem=None,
    )

    report = {
        "added_artists": int(len(added_artists_df)),
        "target_songs": int(len(targets_df)),
        "existing_discovery_rows": int(len(existing_rows)),
        "supplement_discovery_rows": int(len(supplement_rows)),
        "final_discovery_rows": int(len(final_rows)),
        "new_download_ids": int(len(new_ids)),
        "attempted_target_keys": int(len(attempted_target_keys)),
    }
    REPORT_JSON.write_text(json.dumps(report, indent=2), encoding="utf-8")
    log(f"Saved␣report␣to␣{REPORT_JSON.name}")
```

Table 5: Saved Billboard supplement run report

Metric	Value
Added artists in supplement	206
Target songs in executed run report	1,516
Existing discovery rows before supplement	70,496
Rows added by supplement	13
Discovery rows after merge	70,504
New raw chord JSON downloads	8
Attempted target keys saved	1,087

Il punto metodologico qui e che il supplemento non sostituisce la discovery base: la estende in modo mirato. Il report di run salvato documenta un caso concreto con 206 artisti aggiunti, 1,516 target song processate nel run salvato, 13 discovery rows aggiunte e 8 nuovi raw JSON scaricati.

Questa fase spiega anche perche il file country-only corrente contiene 206 artisti in piu rispetto al nucleo originario: il supplemento Billboard ha modificato il perimetro operativo usato a valle.

7 Step 5: costruzione del dataset finale chord-level

7.1 Meccanica del builder finale

Lo script `build_country_only_chords_final.py` parte dal file `Restricted Final v6` e lo estende. La logica e la seguente:

1. legge le discovery rows chord-only e i raw JSON scaricati;
2. costruisce nuove canzoni da appendere al v6;
3. deduplica le nuove canzoni per UG id e per coppia artista-canzone;
4. riattacca i metadati artista dall'universo country-only;
5. crea bridge rows sintetiche per artisti con raw JSON ma senza presenza nel file combinato;
6. applica `num_chord_json_used`;
7. costruisce o carica la cache Billboard year-end;
8. applica i flag chart-level;
9. riempie i `release_year` mancanti;
10. salva CSV e DTA con label.

Snippet chiave: costruzione delle nuove righe song-level da raw JSON.

```
def build_new_song_rows() -> pd.DataFrame:
    discovery_rows = load_discovery()
    v6_df = pd.read_csv(V6_CSV, low_memory=False)
    artist_df = load_artist_universe()
    output_columns = build_output_columns(list(v6_df.columns))
    by_id, by_name = build_artist_metadata_maps(v6_df, artist_df)
    existing_keys = {canonical_song_key(row.artist_name, row.song_name) for row in v6_df.itertuples()}

    grouped = defaultdict(list)
    for row in discovery_rows:
        key = canonical_song_key(row.get("target_artist_name") or row.get("artist_name"), row.get("song_name")
     )
        grouped[key].append(row)

    new_rows = []
    skipped_existing = 0
    missing_raw = 0
    for idx, (key, versions) in enumerate(grouped.items(), start=1):
        if key in existing_keys:
            skipped_existing += 1
            continue
        try:
            base_row, payload, num_json_used = build_enriched_payload(versions)
        except FileNotFoundError:
            missing_raw += 1
            continue
        song_row = flatten_chord_payload(base_row, payload, num_json_used)
        full_row = attach_artist_metadata(song_row, base_row, by_id, by_name, output_columns)
        new_rows.append(full_row)
        if idx % 5000 == 0 or idx == len(grouped):
            log(f"Processed␣song␣groups:␣{idx}/{len(grouped)};␣new␣rows␣accumulated:␣{len(new_rows)}")

    new_df = pd.DataFrame(new_rows, columns=output_columns)
    pre_dedup_count = len(new_df)
    if not new_df.empty:
```

```
        new_df["_artist_norm"] = new_df["artist_name"].map(norm_text)
        new_df["_song_norm"] = new_df["song_name"].map(norm_text)
        new_df.sort_values(["rating", "votes", "id"], ascending=[False, False, False], inplace=True)
        new_df.drop_duplicates(subset=["id"], keep="first", inplace=True)
        new_df.drop_duplicates(subset=["_artist_norm", "_song_norm"], keep="first", inplace=True)
        new_df.drop(columns=["_artist_norm", "_song_norm"], inplace=True)
        new_df = new_df.reindex(columns=output_columns)

    log(f"New␣song␣groups␣discovered:␣{len(grouped)}")
    log(f"Skipped␣because␣already␣in␣v6:␣{skipped_existing}")
    log(f"Skipped␣because␣no␣raw␣JSON␣survived:␣{missing_raw}")
    log(f"Deduplicated␣new␣song␣rows␣removed:␣{pre_dedup_count␣-␣len(new_df)}")
    log(f"New␣songs␣ready␣to␣append:␣{len(new_df)}")
    new_df.to_csv(NEW_SONGS_CSV, index=False)
    return new_df
```

Snippet chiave: cache Billboard e build finale del dataset.

```
def build_chart_cache() -> pd.DataFrame:
    if CHART_CACHE_CSV.exists():
        df = pd.read_csv(CHART_CACHE_CSV)
        if not df.empty and int(df["chart_year"].max()) >= 2025:
            return augment_chart_cache_with_year_end_supplement(df)
        log("Existing␣Billboard␣chart␣cache␣is␣incomplete;␣rebuilding␣it")

    session = requests.Session()
    session.headers.update({"User-Agent": USER_AGENT})

    all_rows = []
    for year in range(1946, 2026):
        rows = scrape_chart_rows_for_year(session, year)
        if rows:
            all_rows.extend(rows)
        if year % 10 == 0 or year == 2025:
            log(f"Billboard␣year-end␣chart␣scrape␣progress␣through␣{year}:␣{len(all_rows)}␣rows")
        time.sleep(0.1)

    chart_df = pd.DataFrame(all_rows)
    if chart_df.empty:
        raise RuntimeError("Could␣not␣build␣Billboard␣country␣year-end␣chart␣cache")

    chart_df = augment_chart_cache_with_year_end_supplement(chart_df)
    chart_df["song_key"] = chart_df["song_name"].map(norm_text)
    chart_df["artist_key"] = chart_df["artist_name"].map(norm_text)
    chart_df.sort_values(["artist_key", "song_key", "chart_year", "position"], inplace=True)
    chart_df.to_csv(CHART_CACHE_CSV, index=False)
    chart_df.to_json(CHART_CACHE_JSON, orient="records", force_ascii=False, indent=2)
    return chart_df


def augment_chart_cache_with_year_end_supplement(chart_df: pd.DataFrame) -> pd.DataFrame:
    if not SUPPLEMENTAL_CHART_CSV.exists():
        return chart_df
    try:
        supplemental_df = pd.read_csv(SUPPLEMENTAL_CHART_CSV, low_memory=False)
    except Exception:
        return chart_df
    if supplemental_df.empty or "source_kind" not in supplemental_df.columns:
        return chart_df
    supplemental_df = supplemental_df[supplemental_df["source_kind"].eq("year_end_wikipedia")].copy()
    if supplemental_df.empty:
        return chart_df
    combined = pd.concat([chart_df, supplemental_df], ignore_index=True, sort=False)
    combined = combined.drop_duplicates(subset=["chart_year", "position", "song_name", "artist_name"])
    return combined
```

```
def apply_chart_variables(df: pd.DataFrame, chart_df: pd.DataFrame) -> pd.DataFrame:
    chart_lookup = {}
    for row in chart_df.itertuples(index=False):
        candidate = (int(row.chart_year), int(row.position))
        keys = {
            canonical_song_key(row.artist_name, row.song_name),
            canonical_song_key(simplify_chart_artist(row.artist_name), clean_chart_song_title(row.song_name)),
        }
        for key in keys:
            current = chart_lookup.get(key)
            if current is None or candidate < current:
                chart_lookup[key] = candidate

    df["billboard_country_year_end_flag"] = 0
    df["billboard_country_year_end_year"] = pd.NA
    df["billboard_country_year_end_pos"] = pd.NA
    df["_artist_chart_key"] = df["artist_name"].map(norm_text)
    df["_song_chart_key"] = df["song_name"].map(norm_text)
    df["_artist_chart_key_simple"] = df["artist_name"].map(simplify_chart_artist).map(norm_text)
    df["_song_chart_key_clean"] = df["song_name"].map(clean_chart_song_title).map(norm_text)

    for idx in df.index:
        keys = [
            (df.at[idx, "_artist_chart_key"], df.at[idx, "_song_chart_key"]),
            (df.at[idx, "_artist_chart_key_simple"], df.at[idx, "_song_chart_key_clean"]),
        ]
        for key in keys:
            if key in chart_lookup:
                chart_year, position = chart_lookup[key]
                df.at[idx, "billboard_country_year_end_flag"] = 1
                df.at[idx, "billboard_country_year_end_year"] = chart_year
                df.at[idx, "billboard_country_year_end_pos"] = position
                break
    df.drop(columns=["_artist_chart_key", "_song_chart_key", "_artist_chart_key_simple", "
     _song_chart_key_clean"], inplace=True, errors="ignore")
    return df


def build_final_dataset() -> pd.DataFrame:
    base_df = pd.read_csv(V6_CSV, low_memory=False)
    output_columns = build_output_columns(list(base_df.columns))
    base_df = base_df.reindex(columns=output_columns)
    discovery_rows = load_discovery()
    if NEW_SONGS_CSV.exists():
        new_df = pd.read_csv(NEW_SONGS_CSV, low_memory=False)
        log(f"Loaded␣cached␣new-song␣rows␣from␣{NEW_SONGS_CSV.name}:␣{len(new_df)}␣rows")
        new_df = deduplicate_new_song_cache(new_df, output_columns)
        new_df.to_csv(NEW_SONGS_CSV, index=False)
    else:
        new_df = build_new_song_rows()
    existing_ids = set(pd.to_numeric(base_df["id"], errors="coerce").dropna().astype(int))
    before_overlap_drop = len(new_df)
    new_df = new_df[~pd.to_numeric(new_df["id"], errors="coerce").fillna(-1).astype(int).isin(existing_ids)].
     copy()
    overlap_removed = before_overlap_drop - len(new_df)
    if overlap_removed:
        log(f"Removed␣new-song␣rows␣whose␣UG␣id␣already␣exists␣in␣v6:␣{overlap_removed}")
    combined = pd.concat([base_df, new_df], ignore_index=True)
    artist_df = load_artist_universe()
    bridge_df = build_artist_bridge_rows(discovery_rows, combined, base_df, artist_df)
    if not bridge_df.empty:
        combined = pd.concat([combined, bridge_df], ignore_index=True)
    combined = apply_num_json_used(combined, discovery_rows)
    combined = apply_chart_variables(combined, build_chart_cache())
    combined = enrich_release_years(combined)
    combined.to_csv(OUTPUT_CSV, index=False)
    return combined
```

```
def build_final_dataset() -> pd.DataFrame:
    base_df = pd.read_csv(V6_CSV, low_memory=False)
    output_columns = build_output_columns(list(base_df.columns))
    base_df = base_df.reindex(columns=output_columns)
    discovery_rows = load_discovery()
    if NEW_SONGS_CSV.exists():
        new_df = pd.read_csv(NEW_SONGS_CSV, low_memory=False)
        log(f"Loaded␣cached␣new-song␣rows␣from␣{NEW_SONGS_CSV.name}:␣{len(new_df)}␣rows")
        new_df = deduplicate_new_song_cache(new_df, output_columns)
        new_df.to_csv(NEW_SONGS_CSV, index=False)
    else:
        new_df = build_new_song_rows()
    existing_ids = set(pd.to_numeric(base_df["id"], errors="coerce").dropna().astype(int))
    before_overlap_drop = len(new_df)
    new_df = new_df[~pd.to_numeric(new_df["id"], errors="coerce").fillna(-1).astype(int).isin(existing_ids)].
     copy()
    overlap_removed = before_overlap_drop - len(new_df)
    if overlap_removed:
        log(f"Removed␣new-song␣rows␣whose␣UG␣id␣already␣exists␣in␣v6:␣{overlap_removed}")
    combined = pd.concat([base_df, new_df], ignore_index=True)
    artist_df = load_artist_universe()
    bridge_df = build_artist_bridge_rows(discovery_rows, combined, base_df, artist_df)
    if not bridge_df.empty:
        combined = pd.concat([combined, bridge_df], ignore_index=True)
    combined = apply_num_json_used(combined, discovery_rows)
    combined = apply_chart_variables(combined, build_chart_cache())
    combined = enrich_release_years(combined)
    combined.to_csv(OUTPUT_CSV, index=False)
    return combined
```

Ci sono due dettagli da sottolineare.

Primo, la variabile `genre` nel file finale e *song-level* e viene letta dal payload UG (`advertising.targeting_song_` non e una proprieta ereditata automaticamente dall'universo artisti. Questo e il motivo per cui, a livello esplorativo, il dataset largo contiene anche brani classificati come `rock`, `pop` o `folk` pur partendo da artisti country-only.

Secondo, il builder finale usa fino a tre JSON chord per canzone e fa *deep backfill* dei payload: non e una semplice scelta del top-1, ma una procedura che cerca di estrarre il massimo contenuto informativo dai migliori file disponibili.

8 Step 6: recupero dei `release_year` mancanti

8.1 Gerarchia di imputazione

Il recupero dei `release_year` e uno dei punti piu delicati dell'intera pipeline. Lo script non usa un solo servizio, ma una gerarchia molto esplicita. L'ordine e:

1. override manuali `WEB_CONFIRMED_RELEASE_YEAR_OVERRIDES`;
2. cache storiche e master cache derivate dal v6 e da cache precedenti;
3. cache dedicate della run country-only finale;
4. lookup artist-level su iTunes;
5. lookup artist-level su MusicBrainz recordings;
6. lookup artist-level su MusicBrainz releases;
7. lookup artist-level su Deezer;

8. lookup song-bundle su iTunes partendo dalle canzoni ancora mancanti;
9. fallback song-level: Discogs, poi MusicBrainz, poi Wikidata, poi Wikipedia.

Table 6: Release-year caches consulted by the final chord builder

Release-year cache file
artist_release_track_cache_country_only_2026_04_08.json
release_years_cache.json
release_years_cache_country_only_chords_final_2026_04_08.json
release_years_cache_discogs.json
release_years_cache_fuzzy.json
release_years_cache_internet.json
release_years_cache_restricted_final_v6.json
release_years_cache_wiki.json
artist_release_track_cache_country_only_2026_04_08.json

Snippet chiave: enrichment dei release year.

```
def enrich_release_years(df: pd.DataFrame) -> pd.DataFrame:
    cache_only = os.environ.get("SOC_RELEASE_YEAR_CACHE_ONLY") == "1"
    dedicated_cache = load_json(RELEASE_CACHE_FILE)
    artist_track_cache = load_json(ARTIST_RELEASE_TRACK_CACHE_FILE)
    master_cache = build_master_release_cache(pd.read_csv(V6_CSV, low_memory=False))
    normalized_master = {normalize_key(key): value for key, value in master_cache.items()}

    df["release_year"] = pd.to_numeric(df["release_year"], errors="coerce")
    df["_artist_norm"] = df["artist_name"].map(normalize_space)
    df["_song_norm"] = df["song_name"].map(norm_text)

    manual_fills = 0
    for idx in df.index[df["release_year"].isna()]:
        key = (normalize_space(df.at[idx, "artist_name"]), normalize_space(df.at[idx, "song_name"]))
        year = WEB_CONFIRMED_RELEASE_YEAR_OVERRIDES.get(key)
        if is_valid_year(year):
            df.at[idx, "release_year"] = int(year)
            dedicated_cache[canonical_release_key(*key)] = int(year)
            manual_fills += 1

    cache_fills = 0
    for idx in df.index[df["release_year"].isna()]:
        key = canonical_release_key(df.at[idx, "artist_name"], df.at[idx, "song_name"])
        if key in master_cache:
            df.at[idx, "release_year"] = master_cache[key]
            cache_fills += 1
            continue
        nkey = normalize_key(key)
        if nkey in normalized_master:
            df.at[idx, "release_year"] = normalized_master[nkey]
            cache_fills += 1
            continue
        cached_value = dedicated_cache.get(key)
        if is_valid_year(cached_value):
            df.at[idx, "release_year"] = int(float(cached_value))
            cache_fills += 1
    log(f"Rows␣filled␣from␣historical␣release␣caches:␣{cache_fills}")
    if manual_fills:
        log(f"Rows␣filled␣from␣web-confirmed␣manual␣release␣overrides:␣{manual_fills}")

    remaining = df[df["release_year"].isna()].copy()
    if remaining.empty or cache_only:
```

```
    save_release_caches(dedicated_cache, artist_track_cache)
    if cache_only:
        log(f"Cache-only␣release-year␣mode␣enabled;␣rows␣still␣missing:␣{int(df['release_year'].isna().sum
 ())}")
    df.drop(columns=["_artist_norm", "_song_norm"], inplace=True, errors="ignore")
    return df

session = requests.Session()
session.headers.update({"User-Agent": USER_AGENT})

artist_order = (
    remaining.groupby("_artist_norm")
    .size()
    .sort_values(ascending=False)
    .index.tolist()
)
processed_artists = 0
for artist_name in artist_order:
    group = remaining[remaining["_artist_norm"] == artist_name]
    songs_by_norm = {norm_text(song): song for song in group["song_name"].tolist()}
    artist_mbids = group.get("musicbrainz_mbid", pd.Series(dtype="object")).dropna().astype(str).str.strip
 ()
    artist_mbids = artist_mbids[artist_mbids.ne("")]
    artist_mbid = artist_mbids.iloc[0] if not artist_mbids.empty else ""
    artist_missing_start = int(df[df["release_year"].isna() & df["_artist_norm"].eq(artist_name)]["
 song_name"].nunique())
    log(f"Release-year␣artist␣start:␣{artist_name}␣with␣{artist_missing_start}␣missing␣songs")
    artist_query_count = 0

    artist_cache_key = artist_mbid or artist_name
    cached_artist_tracks = artist_track_cache.get(artist_cache_key, {})
    if isinstance(cached_artist_tracks, dict):
        for song_norm, year in cached_artist_tracks.items():
            if song_norm not in songs_by_norm or not is_valid_year(year):
                continue
            mask = df["release_year"].isna() & df["_artist_norm"].eq(artist_name) & df["_song_norm"].eq(
 song_norm)
            df.loc[mask, "release_year"] = int(year)
            dedicated_cache[canonical_release_key(artist_name, songs_by_norm[song_norm])] = int(year)

    itunes_found = year_from_artist_itunes(session, artist_name, songs_by_norm)
    for song_norm, year in itunes_found.items():
        mask = df["release_year"].isna() & df["_artist_norm"].eq(artist_name) & df["_song_norm"].eq(
 song_norm)
        df.loc[mask, "release_year"] = year
        song_title = songs_by_norm[song_norm]
        dedicated_cache[canonical_release_key(artist_name, song_title)] = year

    mb_artist_candidates = df[df["release_year"].isna() & df["_artist_norm"].eq(artist_name)]
    if not mb_artist_candidates.empty:
        mb_artist_found = year_from_artist_musicbrainz(session, artist_name, songs_by_norm, artist_mbid=
 artist_mbid)
        for song_norm, year in mb_artist_found.items():
            mask = df["release_year"].isna() & df["_artist_norm"].eq(artist_name) & df["_song_norm"].eq(
 song_norm)
            df.loc[mask, "release_year"] = year
            song_title = songs_by_norm[song_norm]
            dedicated_cache[canonical_release_key(artist_name, song_title)] = year
        if mb_artist_found:
            artist_track_cache.setdefault(artist_cache_key, {}).update({k: int(v) for k, v in
 mb_artist_found.items() if is_valid_year(v)})

    mb_release_candidates = df[df["release_year"].isna() & df["_artist_norm"].eq(artist_name)]
    if not mb_release_candidates.empty:
        mb_release_found = year_from_artist_musicbrainz_releases(session, artist_name, songs_by_norm,
 artist_mbid=artist_mbid)
        for song_norm, year in mb_release_found.items():
```

```
            mask = df["release_year"].isna() & df["_artist_norm"].eq(artist_name) & df["_song_norm"].eq(
song_norm)
            df.loc[mask, "release_year"] = year
            song_title = songs_by_norm[song_norm]
            dedicated_cache[canonical_release_key(artist_name, song_title)] = year
        if mb_release_found:
            artist_track_cache.setdefault(artist_cache_key, {}).update({k: int(v) for k, v in
mb_release_found.items() if is_valid_year(v)})

    deezer_candidates = df[df["release_year"].isna() & df["_artist_norm"].eq(artist_name)]
    if not deezer_candidates.empty:
        deezer_found = year_from_artist_deezer(session, artist_name, songs_by_norm)
        for song_norm, year in deezer_found.items():
            mask = df["release_year"].isna() & df["_artist_norm"].eq(artist_name) & df["_song_norm"].eq(
song_norm)
            df.loc[mask, "release_year"] = year
            song_title = songs_by_norm[song_norm]
            dedicated_cache[canonical_release_key(artist_name, song_title)] = year

    still_missing = df[df["release_year"].isna() & df["_artist_norm"].eq(artist_name)][["artist_name", "
song_name"]].drop_duplicates()
    still_missing_norms = {norm_text(song): song for song in still_missing["song_name"].tolist()}
    for row in still_missing.itertuples(index=False):
        artist_query_count += 1
        bundle_found = year_from_song_itunes_bundle(session, row.artist_name, row.song_name,
still_missing_norms)
        if not bundle_found:
            if artist_query_count % 50 == 0:
                save_release_caches(dedicated_cache, artist_track_cache)
                missing_now = int(df["release_year"].isna().sum())
                log(f"Release-year␣artist␣progress:␣{artist_name};␣song␣queries␣{artist_query_count};␣
missing␣rows␣now:␣{missing_now}")
            continue
        for song_norm, year in bundle_found.items():
            mask = df["release_year"].isna() & df["_artist_norm"].eq(artist_name) & df["_song_norm"].eq(
song_norm)
            df.loc[mask, "release_year"] = year
            song_title = still_missing_norms[song_norm]
            dedicated_cache[canonical_release_key(artist_name, song_title)] = year
        if artist_query_count % 25 == 0:
            save_release_caches(dedicated_cache, artist_track_cache)
            missing_now = int(df["release_year"].isna().sum())
            log(f"Release-year␣artist␣progress:␣{artist_name};␣song␣queries␣{artist_query_count};␣missing␣
rows␣now:␣{missing_now}")
        still_missing = df[df["release_year"].isna() & df["_artist_norm"].eq(artist_name)][["artist_name",
 "song_name"]].drop_duplicates()
        still_missing_norms = {norm_text(song): song for song in still_missing["song_name"].tolist()}
        if still_missing.empty:
            break

    still_missing = df[df["release_year"].isna() & df["_artist_norm"].eq(artist_name)][["artist_name", "
song_name"]].drop_duplicates()
    for row in still_missing.itertuples(index=False):
        artist_query_count += 1
        key = canonical_release_key(row.artist_name, row.song_name)
        year = year_from_song_discogs(session, row.artist_name, row.song_name)
        if year is None:
            year = year_from_song_musicbrainz(session, row.artist_name, row.song_name, artist_mbid=
artist_mbid)
        if year is None:
            year = year_from_song_wikidata(session, row.artist_name, row.song_name)
        if year is None:
            year = year_from_song_wikipedia(session, row.artist_name, row.song_name)
        dedicated_cache[key] = year
        if is_valid_year(year):
            mask = df["release_year"].isna() & df["_artist_norm"].eq(normalize_space(row.artist_name)) &
df["_song_norm"].eq(norm_text(row.song_name))
            df.loc[mask, "release_year"] = int(year)
```

```
            if artist_query_count % 25 == 0:
                save_release_caches(dedicated_cache, artist_track_cache)
                missing_now = int(df["release_year"].isna().sum())
                log(f"Release-year␣artist␣progress:␣{artist_name};␣song␣queries␣{artist_query_count};␣missing␣
    rows␣now:␣{missing_now}")

        processed_artists += 1
        artist_missing_end = int(df[df["release_year"].isna() & df["_artist_norm"].eq(artist_name)]["song_name
    "].nunique())
        log(f"Release-year␣artist␣done:␣{artist_name};␣remaining␣missing␣songs␣for␣artist:␣{artist_missing_end
    }")
        if processed_artists % 10 == 0 or processed_artists == len(artist_order):
            save_release_caches(dedicated_cache, artist_track_cache)
            missing_now = int(df["release_year"].isna().sum())
            log(f"Release-year␣enrichment␣artists␣processed:␣{processed_artists}/{len(artist_order)};␣missing␣
    rows␣now:␣{missing_now}")

    save_release_caches(dedicated_cache, artist_track_cache)
    df.drop(columns=["_artist_norm", "_song_norm"], inplace=True, errors="ignore")
    return df
```

Questa parte va letta come una strategia di minimizzazione dei costi e massimizzazione della precisione:

- prima si prova a riusare conoscenza gia salvata;
- poi si interroga per artista, che e piu efficiente quando mancano molte canzoni dello stesso autore;
- solo alla fine si scende al song-level, che e piu costoso e piu rumoroso;
- durante il loop lo script salva continuamente cache dedicate, cosi la run e riprendibile.

9 Replica cold-start e replica da snapshot

9.1 Cold-start reale

Per rifare davvero la pipeline da capo, la sequenza concettuale e:

1. costruire i dataset artista base con `build_country_artists_dataset.py`;
2. fare top-up del country-only con `enrich_artist_universe_missing_metadata.py -country-only-direct`;
3. lanciare `country_only_ultimate_guitar.py discover`;
4. controllare copertura con `country_only_ultimate_guitar.py plan`;
5. scaricare i raw JSON con `country_only_ultimate_guitar.py download`;
6. eseguire `supplement_billboard_country_chords.py`;
7. costruire il dataset finale con `build_country_only_chords_final.py`.

I comandi pratici, dal root del progetto, sono:

```
python3 execution/step4_country_artists/build_country_artists_dataset.py
python3 execution/step4_country_artists/enrich_artist_universe_missing_metadata.py --country-only-direct

python3 execution/step1_download/country_only_ultimate_guitar.py discover --max-pages 20 --checkpoint-every 10
      --workers 3
python3 execution/step1_download/country_only_ultimate_guitar.py plan
```

```
python3 execution/step1_download/country_only_ultimate_guitar.py download --kind both --workers 3 --min-delay
     0.6 --max-delay 1.6

python3 execution/step1_download/supplement_billboard_country_chords.py --max-pages-artist 20 --max-pages-song
      3 --workers 3 --min-delay 0.6 --max-delay 1.6

python3 execution/step2_digitalize/build_country_only_chords_final.py
```

9.2 Replica assistita da snapshot

Il repository contiene anche wrapper di replica che servono a due funzioni:

- ripristinare output consolidati se non si vuole o non si puo rifare una run live;
- aggiornare e ricreare i replication package con codice, direttive e output correnti.

Lo script piu importante in questa famiglia e `run_full_replication.py`, che per la parte country artists decide se:

1. usare gli output gia presenti nel root del progetto;
2. ripristinare uno snapshot bundled;
3. forzare un full rebuild se richiesto con `-full-rebuild`.

Snippet chiave: wrapper di replica full.

```
def run_country_pipeline(project_root: Path, package_root: Path, *, full_rebuild: bool) -> None:
    if country_outputs_exist(project_root) and not full_rebuild:
        print("Country-artist␣final␣outputs␣already␣present;␣skipping␣rebuild␣and␣snapshotting␣existing␣
     deliverables.", flush=True)
        return
    if not full_rebuild and restore_bundled_country_snapshot(project_root, package_root):
        return
    packaged_builder = package_root / "code" / "python" / "step4_country_artists" / "
     build_country_artists_dataset.py"
    builder = project_root / "execution" / "step4_country_artists" / "build_country_artists_dataset.py"
    if not builder.exists() and packaged_builder.exists():
        builder = packaged_builder
    env = os.environ.copy()
    env["SOC_PROJECT_ROOT"] = str(project_root)
    env["SOC_COUNTRY_SCHEMA_PATH"] = str(package_root / "docs" / "directives_snapshot" / "
     country_artists_schema.csv")
    print("\n>>>␣python3", str(builder.relative_to(project_root) if builder.is_relative_to(project_root) else
     builder), flush=True)
    subprocess.run(["python3", str(builder)], cwd=str(project_root), check=True, env=env)
    fallback_marker = project_root / "data" / "processed_datasets" / "country_artists" / "intermediate" / "
     _bundled_fallback_used.txt"
    if full_rebuild and fallback_marker.exists():
        print("Bundled␣intermediate␣fallback␣was␣used␣during␣full␣rebuild;␣restoring␣canonical␣bundled␣final␣
     outputs.", flush=True)
        restore_bundled_country_snapshot(project_root, package_root)

def main() -> None:
    script_path = current_script_path()
    project_root, package_dirname, full_rebuild = parse_runtime_args(sys.argv)
    package_root = package_root_from_script(script_path, project_root, package_dirname)

    print(f"Project␣root:␣{project_root}", flush=True)
    print(f"Replication␣package␣target:␣{package_root}", flush=True)
    print(f"Full␣rebuild␣mode:␣{full_rebuild}", flush=True)

    ensure_structure(package_root)
    run_country_pipeline(project_root, package_root, full_rebuild=full_rebuild)
```

```
    copy_country_outputs(project_root, package_root)
    copy_python_tree(project_root, package_root)
    copy_directives(project_root, package_root)
    copy_replication_assets(project_root, package_root)

    print("Replication␣pipeline␣completed␣successfully.", flush=True)
```

I wrapper aggiuntivi fanno invece snapshot o restore di componenti specifiche:

- `run_country_songs_replication.py`: output upstream dei song files e release-year caches;
- `run_country_merge_v6_replication.py`: bridge `Restricted Final v6`;
- `run_artist_universe_replication.py`: snapshot e restore del solo universo artista.

La distinzione metodologica e importante: una replica "da snapshot" non e la stessa cosa di una ricostruzione *live*. La prima verifica la portabilita del pacchetto; la seconda verifica la robustezza della pipeline di costruzione.

10 Come leggere correttamente il rapporto tra country-only, adjacent e dataset finale

Una possibile fonte di confusione e la presenza simultanea di tre oggetti:

1. `country-only`: perimetro principale usato per la discovery UG e per il dataset finale qui studiato;
2. `adjacent-only`: perimetro esteso, utile per coverage, confronto e archivio, ma non usato direttamente per il file chord-level country-only;
3. `country plus adjacent`: unione dei due, utile come universo allargato o come prodotto documentale separato.

In altre parole, le *adjacent* servono a costruire un perimetro esteso degli artisti, ma il dataset `Sound_of_Culture_Country_CountryOnly_Chords_Final_2026_04_08` viene costruito sull'universo `artist_universe_country_only.csv`. Il fatto che alcuni brani nel dataset largo non abbiano `genre == country` non contraddice questo punto: dipende dal fatto che il genere e letto a livello di canzone dal payload UG.

11 Conclusione operativa

Se dovessi spiegare la pipeline in una frase sola, direi cosi: il progetto costruisce prima un'identita affidabile degli artisti, poi usa quella identita per scaricare e pulire i contenuti musicali, poi completa i missing piu delicati con una gerarchia di fonti strutturate e fallback documentati, e infine impacchetta sia gli output finali sia le cache necessarie per rendere la replica ripetibile.

Le implicazioni pratiche sono tre:

- per rifare il lavoro da zero bisogna pensare la replica come pipeline multi-step, non come singolo script;
- per ridurre tempi e fragilita conviene conservare checkpoint intermedi e cache, specialmente per `release_year`;
- per valutare il dato finale bisogna sempre distinguere tra perimetro degli artisti e classificazione song-level del payload UG.

A Dove trovare il codice completo

Questo report include soltanto snippet funzionali. Le copie integrali degli script principali sono salvate nella cartella `pipeline_code/` del pacchetto del report, mentre gli artefatti di supporto usati per costruire il report sono salvati in `pipeline_assets/`.